CURRICULUM COMPANION : TECHNICAL LEARNING MODULES & FREE COURSES

Week 3 Learning Materials : Deep Learning & Cortex Core Models

NumPy Backpropagation, PyTorch Modules, Transfer Learning, ONNX & 100% Free Courses

TARGET AUDIENCE	ESTIMATED READING TIME	COURSE COST	PRIMARY OBJECTIVE
ML Interns (Product Phase)	45-55 Minutes	100% Free (Zero Paywalls)	First-Principles Deep Learning

01 Computational Graphs, Backpropagation & Gradient Checking

MODULE 1

Modern deep learning frameworks compute analytical gradients automatically via computational DAGs. To debug numerical instabilities and loss explosions, engineers must understand the underlying multivariate chain rule. Before using PyTorch, you verify your analytical gradients against two-sided finite difference numerical approximations:

```
# Two-sided numerical gradient checking formula
def check_numerical_gradient(f, w, grad_analytical, eps=1e-7):
    grad_num = np.zeros_like(w)
    for idx in np.ndindex(w.shape):
        orig = w[idx]; w[idx] = orig + eps; l_plus = f(w)
        w[idx] = orig - eps; l_minus = f(w); w[idx] = orig
        grad_num[idx] = (l_plus - l_minus) / (2 * eps)
    rel_error = np.linalg.norm(grad_num - grad_analytical) / (np.linalg.norm(grad_num) +
np.linalg.norm(grad_analytical))
    return rel_error < 1e-5
```

02 PyTorch Module Architecture & Training Discipline

MODULE 2

When migrating from NumPy to PyTorch, enforce clean abstraction boundaries. Subclass **`torch.nn.Module`**, separate forward activations from loss criteria, and always call **`optimizer.zero_grad()`** before **`loss.backward()`**:

CORE PYTORCH ENGINEERING INVARIANTS

1. Train vs Eval Mode: Always invoke `model.train()` during training and `model.eval()` with `torch.no_grad()` during validation to properly toggle Dropout and BatchNorm.
2. Gradient Accumulation: When training on memory-constrained CPUs, simulate larger batch sizes by accumulating gradients over multiple forward passes before calling `optimizer.step()`.
3. Inductive Bias Trade-off: Fully-connected MLPs lack translational invariance and spatial locality. For tabular features, tree-based models (LightGBM) often outperform MLPs unless extensive normalization is applied.

03 Hugging Face Transformers & Intent Classification

MODULE 3

For natural language intent routing in Attenomy Cortex, heavy foundation models are disqualified due to prohibitive latency. Lightweight compact models (**`DistilBERT`** or **`all-MiniLM-L6-v2`**) provide 95% of full BERT performance with a 40% reduction in parameters and 60% speedup:

```
# Tokenization and transformer sequence classification pipeline
from transformers import AutoTokenizer, AutoModelForSequenceClassification
tokenizer = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
model = AutoModelForSequenceClassification.from_pretrained("sentence-transformers/all-MiniLM-L6-v2",
num_labels=4)
tokens = tokenizer(query, padding=True, truncation=True, max_length=128, return_tensors="pt")
logits = model(**tokens).logits
```

04 Vision Layout Classification with MobileNetV3

MODULE 4

Document extraction pipelines require classifying incoming receipts, invoices, and contracts. Fine-tuning a pretrained **`MobileNetV3-Small`** backbone leverages depthwise separable convolutions and inverted residuals to achieve high classification accuracy under 15MB memory footprint:

TRANSFER LEARNING STAGING RECIPE

Stage 1 (Feature Extraction): Freeze convolutional backbone parameters (`param.requires_grad = False`). Replace classifier head with a linear layer matching Cortex document classes. Train for 3-5 epochs.

Stage 2 (Fine-Tuning): Unfreeze top residual blocks. Lower learning rate by 10x (e.g. from 1e-3 to 1e-4) with AdamW to avoid catastrophic forgetting of low-level visual features.

05 ONNX Runtime Optimization & CPU Latency SLA

MODULE 5

Attenomy Cortex production microservices enforce a strict sub-50ms CPU inference latency budget. Exporting PyTorch computation graphs to Open Neural Network Exchange (ONNX) format eliminates Python GIL contention, fuses operations, and optimizes memory layouts:

```
# Export PyTorch model to optimized ONNX graph for sub-50ms CPU execution
torch.onnx.export(model, dummy_input, "cortex_router.onnx", input_names=["input_ids"], output_names=
["logits"],
    dynamic_axes={"input_ids": {0: "batch_size", 1: "seq_len"}}, opset_version=14)
session = onnxruntime.InferenceSession("cortex_router.onnx", providers=["CPUExecutionProvider"])
```

06 Curated 100% Free Deep Learning Courses & Labs

ZERO PAYWALLS

All recommended courses and technical resources below are completely free, open-access, and require zero credit cards or paid subscriptions:

RESOURCE / COURSE	INSTRUCTOR / ORG	KEY TOPICS COVERED	ACCESS FORMAT
Neural Networks: Zero to Hero	Andrej Karpathy	Micrograd backpropagation, autograd, PyTorch tensors, language modeling	Free YouTube Playlist
Practical Deep Learning for Coders	fast.ai (Jeremy Howard)	PyTorch vision, transfer learning, tabular embeddings, deployment	100% Free Course & Book
Hugging Face NLP Course	Hugging Face Team	Transformer pipelines, fine-tuning, tokenization, ONNX optimization	Free Interactive Labs
Deep Learning Specialization (Free Audit)	Andrew Ng (DeepLearning.AI)	Neural network math, vectorization, hyperparameter tuning, Adam	Free Audit on Coursera
CS231n / CS224N Lecture Notes	Stanford University	Convolutional architectures, attention mechanisms, loss surfaces	Open Public Web Notes

07 Week 3 Reading & Hands-On Study Checklist

VERIFICATION CHECKLIST

- ☐ **Karpathy Video 1:** Watch "Building micrograd" and trace backpropagation on paper.
- ☐ **Hugging Face Chapter 1-3:** Read pipeline abstractions, tokenizers, and dataset mapping.
- ☐ **ONNX Runtime Guide:** Learn CPU execution provider graph optimizations and thread pooling.
- ☐ **Gradient Check:** Verify analytical gradients against finite differences with error < 1e-5.
- ☐ **Torchvision Docs:** Review MobileNetV3 architecture and depthwise separable convolution math.
- ☐ **MLflow / W&B:** Set up experiment tracking to log training loss, validation accuracy, and parameters.

08 Attenomy Engineering Standard : Grounded Optimization

QUALITY GATE

Evidence-Driven Rigor: Never claim a model is production-ready based on training accuracy alone. Evaluate out-of-vocabulary queries, measure CPU p95 latency under concurrent load, and maintain reproducible random seeds across all training splits.
Combating the 10-Minute AI Trap: Prompting an LLM to generate code is fast, but real engineering lies in specifying constraints, defensive testing, and knowing the mathematical underpinnings. Be prepared to defend your architecture in Friday's 1-on-1 review.